A Robust Transformer Based Sentiment Analysis System

Umakanth N[1], Muhesh Prabhu S, Pugal Abishek A[3]
[1]AP(Sl.Grade), [2,3] UG Student
[1,2,3]*Department of Computer Science and Engineering, Mepco Schlenk Engineering College, Sivakasi 626 005, Tamil Nadu, India*
E-mail: umakanthn@mepcoeng.ac.in.com [1], muhesh30568_bcs27@mepcoeng.ac.in[2],
pugalabishek1042006_bcs27@mepcoeng.ac.in[3]

Abstract

In Sentiment Analysis, it is still challenging to detect sentiment from movie reviews written by users because often opinions are negated, expressed with sarcasm or contrast and also these are dispersed across lengthy sections of text. This paper presents a Hybrid Classifier (A Robust Transformer Based Sentiment Analysis System), that combines an Ordered-Neuron LSTM (ON-LSTM) decoder with a pretrained DeBERTa-v3 encoder. Then the encoder passes the token representations, which are also context-aware, to the ON-LSTM which decodes them back in order, but which has the memory organized hierarchically, with the neurons that change slowly holding the long-range polarity signals, and the ones that change quickly holding the local phrasing. An additive attention-pooling layer then further compresses the sequence to a single vector that is then passed to a small fully connected network to generate class probabilities. To improve robustness, the word-embedding matrix is perturbed in a direction determined by the loss gradient (Fast Gradient Method, FGM) and the main hyper-parameters are optimized with Optuna using a Tree-structured Parzen Estimator sampler and median pruner. The model is evaluated on the 50,000-review IMDB corpus in a 70/20/10 stratified split and the evaluation is done using the accuracy, precision, recall and F1-score. On the 10,000-review test set that was held out the fine-tuned model achieves an accuracy of 95.36 per cent and an F1-score of 95.62 per cent.

Keywords: sentiment analysis, DeBERTa, ordered-neuron LSTM, attention pooling, adversarial training, and hyper-parameter optimisation.

I. INTRODUCTION

Online platforms receives large amount of data regarding reviews and immediately predict whether the review is positive or negative, which helps in recommendation systems, reputation control and market analysis only which can be only analysed with the movie reviews. A binary classification but not easy. A review can say the film is good and then go on to criticize the film, or he can give something negative comments and then throw in an ironic rebuttal, or he can turn a positive statement to a negative a few words later. A mechanism for transferring the overall attitude over a long passage and rich lexical context are needed for a classifier [1-2].

Industry is now offering answers to the first requirement: the transformer-based encoder, which is introduced by industry. Inspired by the BERT family, Models are trained to learn bidirectional context from large unlabeled text and is then transferred well to perform classification, but DeBERTa has a couple of extra elements or features to further enhance sample efficiency: it uses two content vectors and two position vectors for each token and then it uses disentangled attention computation for pre-training to boost sample efficiency in its third version, replaced-token-detection pretraining [3]. However, even if a review is not explicitly modelled as a change in sentiment of each clause, it can be summarized in one step of the output of the pooled encoder.

In terms of Recurrent networks which provides cumulative-softmax master gates, the ordered-neuron LSTM learns an ordering over the cell's neurons, thereby some of them are updated periodically, and then retain high-level information, whereas others are updated frequently [4]. But it can be not a perfect fit for reviews usually containing sentences within paragraphs, this is a natural fit for sentences containing clauses.

Neural text classifiers are also get easily affected by small changes in the input. When a text model is applied in a recurrent manner, adversarial training has been found to regularise such models and then enhance generalisation performance [5]. Finally, learning rates and drop-out, hidden sizes etc., parameters are closely related to performance, and expensive to hand-tune or fine-tune, and can be systematic through automated search frameworks such as Optuna hyper-parameter optimizer [6].The following is an overview of the contributions related to the work:

1) Review-level sentiment classification with hybrid architecture of stacking an ON-LSTM decoder and an attention-pooling head on top of an encoder named DeBERTa-v3.

2) An adversarial training scheme based on FGM applied to word embeddings, along with the use of mixed-precision training and gradient clipping for stability.

3) A hyper-parameter optimisation procedure was conducted on a small subset of the data using the hyper-parameter optimiser: Optuna with Tree Parzen Estimator sampling and median pruning.

4) A reproducible experimental procedure on the IMDB dataset with train/validation/test split 70/20/10 and a standard set of classification metrics.

II. Related Work

A. Sentiment classification on movie reviews

The IMDB is a standard dataset used to learn polarity classification, It is first addressed by word-vector models which carry both sentiment and semantic information for each word [7]. Further researches are on these are related to review corpus which has been shifted from hand-crafted lexicon to distributed representation [15]. Standard CoreNLP sentiment annotator [9] which uses a rule or pipeline based toolkit with a pre-made sentiment built on hand-crafted linguistic features, and is easy to interpret because it is feature and rule based. SentiStrength [10] which assigns strength scores to words and combines them into a single document level score and it is designed mainly foe short, informal writing rather than long-form reviews. SentiWordNet 3.0 [11] which provides scores for WordNet synsets used as a lexical resource for scoring. SMSR manifold based sparse representation [12] used for text classification which uses IMDB corpus but yet receives F1 score of 0.9346. Neural probabilistic language model proposed by the authors Bengio et al [14], and their system is trained with language-modelling loss function to learn distributed word representations, it is one of the earliest neural approaches to learn word representation directly from the data instead of hand engineering. Word2vec [15] learns dense word vectors from local context windows over web scale text, and the system uses web scale unlabelled text as dataset, which made pretrained word embeddings a standard, reusable input for almost any downstream text classifier. CNN for sentence classification proposed by Kim et al. [16] applies convolutional filters with different widths fixed over sentence embeddings. The system uses standard sentence-classification benchmarks of time as dataset and shows that simple convolution over pretrained embeddings, with no recurrence is enough for competitive sentence classification. Recursive neural tensor network proposed by Socher et al. [17] uses a recurrent neural tensor network trained on the sentiment treebank to compose phrase level meaning. As the system uses sentiment Treebank as dataset, it explicitly modelled compositional effects such as negation, which a simple bag-of embeddings classifier cannot capture it.

B. Pretrained language encoders

Pretrained language models learn contextual representations from large-scale corpora and have become an important methods for text classification. BERT [8] which uses masked-language model pretraining with bidirectional context, and it uses BooksCorpus and English Wikipedia as dataset. The unique recipe is its bidirectional masked-language, which is default for the pretrained encoders that are later built on. ELMo [18] learns contextual word embeddings from a bidirectional encoder model, and embeds depending on the full sentence rather than being a fixed lookup table. The system uses large unlabelled text corpus for language-model pretraining, and it is one of the first approach to produce context dependent embeddings. GPT [19] uses a transformer-based decoder trained for next-token prediction, and then it is fine-tuned for classification task. Dataset used is large unlabelled text corpus, which demonstrated a purely generative, unidirectional transformer could be fine-tuned effectively. ULMFiT [20] proposed discriminative fine-tuning and unfreezing of layers during fine-tuning. It had adapted a pretrained language model to a new task without retraining from scratch. RoBERTa [21] uses the same masked-language model but with long training, large batchs and dynamic masking. The dataset is same as the BERT model. Proposing that BERT was under-trained but most of the gain is from training not a new architecture. XLNet [22] uses permutation language modelling that recovers bidirectional context without explicitly masking the tokens. Thus avoiding the train or fine tune mismatch caused by masking. ALBERT [23] uses reduced parameter count with factorised embedding parameterisation and cross-layer parameter sharing, which targets the model size or efficiency rather than the accuracy along. ERNIE [24] et al. incorporates explicit entities into a pretraining task and it injects external knowledge into pretraining rather than relying only on raw text statistics. DeBERTa [25] uses disentangled attention mechanism with vectors namely context and positional vectors. Deberta-v3 [3] replaces masked-language modelling with a discriminative replaced token detection objective (generator + discriminator) and this provides more sample efficient than masked-language modelling. Fine-tuning strategy study proposed by Sun et al. [26] and they had investigated fine-tuning strategies for BERT and recommended layer-wise learning decay, which is widely reused in later fine-tuning system.

C. Recurrent models with structural bias

Recurrent networks gives exactly the temporal perspective. Later, sequence-to-sequence learning with LSTMs [30] and GRU [31] encodes variable length sentence into a fixed size vector, which gave the field a working way to compress a whole sequence into one vector representation. Semi-supervised LSTM pretraining [32] pre-trains an LSTM-based sequence encoder on unlabelled data before task-specific training, which uses unlabelled text corpus for pretraining, then a labelled task-specific set. The system showed the benefit of initialising a recurrent encoder from unlabelled data before fine-tuning on the target task. Attention mechanisms or self-attention [33], [34] lets a network to attend the most informative positive in a sequence in a sequence rather than relying only on the final hidden state. This removes the bottleneck of summarising an entire sequence into a single final hidden state. Gated recurrent network for documents [35], proposed by Tank et al. uses gated recurrent network to encode sentences which models documents hierarchically rather than as one flat sequence. Hierarchical attention networks proposed by Yang et al. [36] learns words and sentence matters most for final label when using two levels of attention. ON-LSTM [4] splits hidden units into chunks and regulates them with cumulative-softmax master gates, producing an implicit ordering of memory units.

D. *Adversarial Training and Hyper-parameter Search*

Fast Gradient Sign Method proposed by Goodfellow et al. [39] perturbs the input along the gradient direction of loss to generate an adversarial example, which later adapted for text

embeddings. FGM – embedding-level perturbation [5] which adapts the fast-gradients idea to text by perturbing the continuous word-embedding vectors instead of discrete input token, used as a training-time regulariser. FreeLB [40] uses multi step projected gradient training, where the embedding perturbation is repeatedly projected back onto a normalisation ball, which made stonger robustness than single-step FGM, but at noticeably higher computational cast per step. Random search vs. grid search, proposed by Bergstra and Bengio [41] compares random sampling of hyper-parameters against grid search. They showed random search is more efficient than grid search in high dimensional configuration spaces. Tree-structured Parzen Estimator (TPE) [42] models the promising and unpromising regions of the hyper-parameter search space using two separate densities, built using validation F1 scores, which uses the outcome of earlier trails to decide which region to sample next, instead of sampling blindly. Hyperband [43] allocates the training budget across trails through successive halving, stopping unpromising runs early, which saves compute by killing clearly bad trails early than running till completion. Optuna [6] integrates samplers such as TPE and pruners such as median pruner into define by run optimisation framework, where the later trails uses the information from the earlier ones.

III. Proposed Methodology

A. Overview

Fig. 1 shows the architecture. The raw IMDB review is cleaned and tokenised, and then entered into a DeBERTa-v3 encoder. The cleaned review is then tokenised and input into the DeBERTa-v3 encoder. The output token vectors from the encoder is then are regularized with dropout and fed into the ON-LSTM decoder consisting of K memory layers. Then form a review vector, by using an attention-pooling layer, followed by a fully connected network and softmax to obtain the probabilities of the required classes. During the training phase, an FGM adversarial training branch distorts the embedding matrix and during a separate search phase, and before all these works, Optuna hyper-parameter optimizer fine tunes the hyper-parameters using TPE with the help of F1 score.

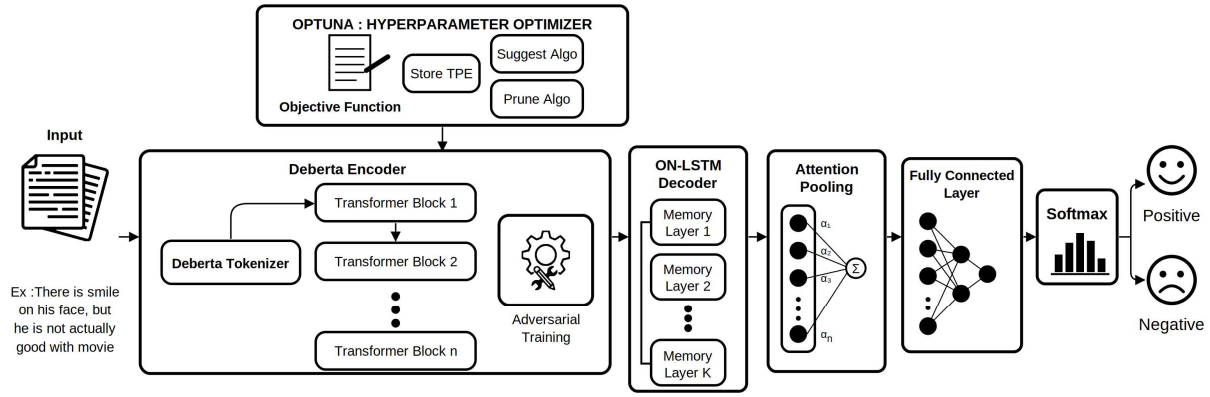


Fig. 1. Overall architecture of the proposed system.

B. Text preprocessing

The review is marked up with noise and they do not convey any sentiment. The four steps of each review are: (i) Decoding of html entities; (ii) replacement of html tags with a single space; (iii) removal of web addresses; and (iv) normalisation of the text to unicode NFKC normalization form and collapsing whitespace characters. This outputs cleaned text which is then tokenised using the DeBERTa-v3 tokenizer that adds the [CLS] and [SEP] markers and truncates sequences to a maximum length of 128 tokens. The padding is added on-the-fly per mini-batch, and a binary attention mask stores the attention positions.

Algorithm 1 Data Pre-processing and Tokenization

Input: IMDB movie-review dataset D, DeBERTa tokenizer T, maximum length L
Output: Encoded train, validation and test sets of (input_ids, _{attention}_mask, label)
for each review r in D
r = HTML-unescape(r) // Remove HTML tags (e.g.
) and URLs from r
r = Unicode NFKC-normalise(r) // Removes Collapse repeated whitespace and strip r
Split D into D_{train} (70 %), D_{test} (20 %) and D_{val} (10 %)
for each split S ∈ {D_{train}, D_{val}, D_{test}}
for each review r in S
tokens = T(r) // sub-word (SentencePiece) tokenization
Add [CLS] at the start and [SEP] at the end
Truncate tokens to L tokens
$X = [x_1, x_2, \dots, x_n]$ // map each token to its vocabulary index
for each mini-batch
Pad every X with [PAD] to the longest sequence in the batch
$M_t = 1$ for real tokens
$M_t = 0$ for padding //Attention Mask
Return encoded D_{train}, D_{val}, D_{test}

C. DeBERTa encoder

Suppose that $X = [x_1, x_2, \dots, x_n]$ is the token sequence of a sample review. It is then converted by the pretrained encoder to a matrix of contextual vectors:

$$H = \text{DeBERTa}(X), \quad H \in \mathbb{R}^{T \times d} \tag{1}$$

where T is the sequence length and d is the hidden size of the encoder (768 for the base model). All encoder weights are fine-tuned, using a small learning rate than the newly added layers because of it had been pre-trained on English language. Dropout is applied to H before it enters the decoder.

Algorithm 2 DeBERTa Encoder (Contextual Encoding)

Input: Token indices X = $[x_1, \dots, x_n]$, attention mask M
Output: Contextual token representations H = $[h_1, \dots, h_n]$, $h_t \in R^{768}$
//Load pre-trained DeBERTa-v3-base weights (12 Transformer blocks, hidden size 768)
for each token x_t
c_t^0 = WordEmbedding(x_t) // content vector; position is handled by relative embeddings P
for each Transformer block l = 1 to 12
for each token pair (i, j)
$A_{ij} = Q_i^c * K_j^c + Q_i^c * K_{i-j}^r + K_j^c * Q_{j-i}^r$
// content-to-content + content-to-position + position-to-content (disentangled attention)
$A_{ij} = \frac{A_{ij}}{\sqrt{3d}}$ // d is the size of query, key vectors
if $M_j = 0$ then $A_{ij} = -\infty$ // mask padding
α = softmax(A)
c^l = LayerNorm($c^{l-1} + \alpha$ V)
c^l = LayerNorm(c^l + FeedForward(c^l))
$h_t = c_t^{12}$ for every token t
Return H

D. ON-LSTM decoder

The decoder reads the sequence H one position at a time. Its distinguishing element is the cumulative softmax, denoted cumax, which converts a score vector into a non-decreasing sequence between 0 and 1:

$$cumax(u)_k = \sum_{j=1}^{k} softmax(u)_j \quad (2)$$

The hidden units are further divided into chunks, and two master gates are computed for each chunk from the current input x_t and previous hidden state h_{t-1}. The master forget gate rises monotonically across each of the neuron orders, whereas the master input gate falls:

$$\tilde{f}_t = cumax(W_{\tilde{f}}\, x_t + U_{\tilde{f}}\, h_t - 1 + b_{\tilde{f}}) \quad (3)$$

$$\tilde{\imath}_t = 1 - cumax(W_{\tilde{\imath}}\, x_t + U_{\tilde{\imath}}\, h_t - 1 + b_{\tilde{\imath}}) \quad (4)$$

Their element-wise product defines the region in which both gates are active:

$$\omega_t = \tilde{f}_t \circ \tilde{\imath}_t \quad (5)$$

This overlap is used to combine the master gates with the standard forget gate f_t and input gate i_t of conventional LSTM:

$$\hat{f}_t = f_t \circ \omega_t + (\tilde{f}_t - \omega_t) \quad (6)$$

$$\hat{\imath}_t = i_t \circ \omega_t + (\tilde{\imath}_t - \omega_t) \quad (7)$$

The standard gates and candidate cell content are computed per neuron, with σ the logistic sigmoid and $\circ$ the Hadamard product:

$$\begin{aligned} f_t &= \sigma(W_f\, x_t + U_f\, h_t - 1 + b_f) \\ i_t &= \sigma(W_i\, x_t + U_i\, h_t - 1 + b_i) \\ o_t &= \sigma(W_o\, x_t + U_o\, h_t - 1 + b_o) \\ \hat{c}_t &= tanh(W_c\, x_t + U_c\, h_t - 1 + b_c) \end{aligned} \quad (8)$$

The memory cell and hidden state are then updated as:

$$c_t = \hat{f}_t \circ c_t - 1 + \hat{\imath}_t \circ \hat{c}_t \quad (9)$$

$$h_t = o_t \circ tanh(c_t) \quad (10)$$

Because of the reviews are in a mini-batch which have different lengths, they need to add padding mask $m_t \in \{0, 1\}$ prevents padded positions from altering the memory. For a padded position, the previous state is simply carried forward, and its output is set to zero:

$$\begin{aligned} h_t &= m_t\, \tilde{h}_t + (1 - m_t) h_{t-1} \\ c_t &= m_t\, \tilde{c}_t + (1 - m_t) c_{t-1} \end{aligned} \quad (11)$$

where $\tilde{h}_t$ and $\tilde{c}_t$ are the freshly computed states from Eqs. (9)–(10). With K > 1 layers, the output sequence of one memory layer becomes the input of the next, with dropout applied between layers. The output of the decoder is the matrix Z = $(z_1, \dots, z_T)$.

Algorithm 3 ON-LSTM Decoder (Hierarchical Sequence Decoding)

Input: Encoder output H = $[h_1, \dots, h_n]$, attention mask M, hidden size d_h, chunk size C, number of memory layers K
Output: Decoded token representations Z = $[z_1, \dots, z_n]$
$n_{chunk} = \frac{d_h}{C}$ // number of neuron chunks
U = H // input of the first memory layer
for each memory layer k = 1 to K
Initialise hidden state $s_0 = 0$ and cell state $c_0 = 0$
for each time step t = 1 to n
gates = $W_{ih}\, U_t + W_{hh}\, s_{t-1}$ + b
// Split gates into master gates ($f, \tilde{\imath}$), one per chunk, standard gates ($o, \hat{c}, i, f$), one per token
f = cumax(f); $\tilde{\imath}$ = 1 − cumax($\tilde{\imath}$) // cumax(x)k = $\Sigma(j \le k)$ $softmax(x)_j$
$\omega = f \circ \tilde{\imath}$ // overlap of master gates
$f = f \circ \omega + (\tilde{f} - \omega)$; $i = i \circ \omega + (\tilde{\imath} - \omega)$
$c_t = f \circ c_t - 1 + i \circ \hat{c}$ // hierarchical memory update
s_t = o ∘ tanh(c_t)
if $M_t = 0$ then keep previous s, c and output o // padding
U = Dropout(outputs s_1 ... s_n) // input of next layer
Z = output sequence of layer K
Return Z

E. Attention pooling and classification

Instead of using the last state, the model learns a weight for each token, which are either a padded and uninformative state, or an informative state. A small feed-forward scorer is first used to calculate a relevance score:

$$e_t = v_\top\, tanh(W_a\, z_t + b_a) \quad (12)$$

The scores are normalised with a softmax which only considers non-padded positions, so that padding receives zero weight, and the review vector is the weighted sum of the decoder outputs:

$$\alpha_t = softmax(e_1, \dots, e_n \quad (13)$$

$$s = \sum_{t=1}^{T} \alpha_t z_t \quad (14)$$

Algorithm 4 Attention Pooling

Input: Decoded representations Z = $[z_1, \dots, z_n]$, attention mask M
Output: Review-level vector s, attention weights $\alpha = [\alpha_1, \dots, \alpha_n]$
for each token t = 1 to n
$u_t = tanh(W_a\, z_t + b_a)$ // projection
$e_t = v^t\, u_t$ // importance score of the token
if $M_t = 0$ then $e_t \leftarrow -\infty$ // padding gets no attention
α = softmax($e_1, \dots, e_n$)
$s = \Sigma\, \alpha_t\, z_t$, for t = 1 to n
Return s, α

After passing through the fully-connected dropout block, it goes through hidden layer with GELU activation, then a dropout block and an output layer, and is finally a softmax to produce class probabilities:

$$p = softmax(W_2\ GELU(W_1\ s + b_1) + b_2) \quad (15)$$

The parameters are trained as a result of minimizing the cross-entropy loss between the set of the predicted labels and the actual labels over a mini-batch of N reviews:

$$L(\theta) = -(\frac{1}{N}) \sum_{i=1}^{N} \log p_i, y_i \quad (16)$$

where θ denotes all trainable parameters and y_i is the true class of review i. The attention weights α_t additionally allow the contribution of each token to be inspected.

Algorithm 5 Fully Connected Layer and Softmax

Input: Review-level vector s ∈ R^{dh}, dropout rate p, true label y (training only)
Output: Class probability ŷ = [$\hat{y}_{Neg}$, $\hat{y}_{Pos}$], predicted class
  s = Dropout(s, p)
  u = GELU($W_1\ s + b_1$) // hidden layer, size $\frac{d_h}{2}$
  u = Dropout(u, p)
  z = $W_2\ u + b_2$ // logits for the 2 classes
  ŷ = softmax(z)
 if training then
  L = $-\Sigma\ yc\ log\ \hat{y}c$ // cross-entropy loss
 predicted class = argmax(ŷ) // 0 = Negative, 1 = Positive
 Return ŷ, predicted class

F. Adversarial training with FGM

An adversarial perturbation is then added to the word-embedding matrix E to make the classifier less sensitive to small changes in the representation of the input..After a normal forward and backward pass produces the gradient g of the loss with respect to E, the embeddings are shifted in the direction that increases the loss, with the size of the shift controlled by ε:

$$r_{adv} = \frac{\varepsilon \cdot g_E}{\|g_E\|_2}\ ,\ g = \nabla E\ L(\theta;\ E) \quad (17)$$

Two rounds of forward and backward passes are made with the disturbed embeddings E + radv from the Adversarial FGM training. The gradients for the clean pass are added to the gradients for the adversarial pass, so the optimiser is minimising the sum of the clean loss and the adversarial loss; the initial embeddings are recovered before the update to the parameters:

$$L_{total} = L(\theta;\ E) + L(\theta;\ E + r_{adv}) \quad (18)$$

If the norm of the gradient became zero or does not exist (as may be the case with half-precision arithmetic), then the perturbation is not performed for this step. If $\varepsilon = 0$, then adversarial training is turned off.

Algorithm 6 Adversarial Training (Fast Gradient Method on Embeddings)

Input: Model parameters θ, mini-batch (X, M, y), perturbation strength ϵ, clip value g_{max}
Output: Updated parameters θ
 for each mini-batch
  L = CrossEntropy($f(X, M; \theta), y$) // clean loss
  Back-propagate L to obtain gradients $\nabla\ \theta$ L
  E = word-embedding matrix of the DeBERTa encoder; $g_E = \nabla$ E L
  Save backup E_{orig} = E
  if $\left\|g_E\right\|_2 \neq 0$ and $\left\|g_E\right\|_2$ is finite then
   $r_{adv} = \frac{\varepsilon \cdot g_E}{\|g_E\|_2}$ // adversarial perturbation
   E = E + r_{adv}
  L_{adv} = CrossEntropy($f(X, M; \theta), y$) // loss on perturbed embeddings
  // Back-propagate L_{adv} (gradients are added to those of L)
  Restore E = E_{orig}
  // Clip the gradient norm to g_{max}
  // Update θ with AdamW and the linear warm-up scheduler
 Return θ

H. Hyper-parameter optimisation with Optuna

Hyper-parameters are optimized by selecting them so as to maximize the validation F1-score of short training runs:

$$\lambda^* = argmax\ \lambda \in \Lambda\ F1_{val}(\lambda) \quad (19)$$

where λ is a hyper-parameter configuration picked from the search space Λ. The search covered ten hyper-parameters: the encoder learning rate (1×10^{-5} to 5×10^{-5}, log scale), the head learning rate (1×10^{-4} to 2×10^{-3}, log scale), weight decay (0 to 0.1), warm-up ratio (0 to 0.15), dropout (0.05 to 0.3), ON-LSTM hidden size {128, 256, 384}, chunk size {8, 16, 32}, FGM strength ε (0.1 to 1.5), the number of ON-LSTM layers {1, 2} and the batch size {16, 32}. A trial was trained on a 10 % subset of the training and validation sets (3,500 and 500 reviews, respectively) for a maximum sequence length of 256 tokens and the score of a trial was defined as its maximum F1-score for validation over the epochs. The results of previous trials are used to suggest new configurations with a TPE sampler (seed 42) [43], and a median pruner (three start-up trials, one warm-up epoch) stops a trial if its intermediate F1-score falls below the median of previous trials at the same epoch. The total of 6 trials was conducted and the optimal configuration was then applied to train the final model on the full training split.

Algorithm 7 Optuna Hyperparameter Controller

Input: Search space Λ = {$lr \in [2\times10^{-5}, 5\times10^{-5}]$, $lr_{head} \in [10^{-4}, 6\times10^{-4}]$, $wd \in [0, 0.1]$, $\rho_{warm} \in [0.06, 0.2]$, $p \in [0.05, 0.3]$, $d_h \in \{128, 256\}$, $C \in \{8, 16, 32\}$, $\varepsilon \in [0.1, 1.0]$}, number of trials N_{trials}, training subset D_s, validation subset D_v, maximum epochs E, random-start trials $N_0 = 4$, pruner trials $N_p = 3$
Output: Best hyperparameter set λ^* = argmax *score*
Initialise trial history $H = \emptyset$
 for each trial n = 1 to N_{trials}
  if $n \leq N_0$ then
   $\lambda_n \sim$ Uniform(Λ) // random start (log-uniform for the learning rates)
  else // TPE: Suggest Algo
   k = min($\lceil 0.1\ |H| \rceil$, 25)
   H_g = k trials of H with the highest score
   $H_b = \frac{H}{H_g}$
   $l(\lambda)$ = Parzen density estimate of H_g;
   $g(\lambda)$ = Parzen density estimate of H_b
    // Draw candidates $\lambda^1, \ldots, \lambda^K \sim l(\lambda)$, with $K = 24$
   $\lambda_n = argmax_k\ (\frac{l(\lambda^k)}{(g(\lambda^k))})$ //maximises the expected improvement

//Build the hybrid model with λ_n
for each epoch $e = 1$ to E
Train the model on D_s with adversarial training (Algorithm 6)
$P = \frac{TP}{TP+FP}$
$R = \frac{TP}{TP+FN}$ on D_v
$\mathrm{F1}_e = \frac{2 \cdot P \cdot R}{P+R}$ // objective function value
Report $\mathrm{F1}_e$ to the study
if $n > N_p$ and $e > 1$ and $\mathrm{F1}_e <$ median$\{\mathrm{F1}e(m) : m = 1 \ldots n-1\}$
then // Median pruner: Prune Algo
Mark trial n as pruned and stop it (its last $F1_e$ is kept as its score)
$score_n = \max_e \ \mathrm{F1}_e$
$H = H \cup \{(\lambda_n, score_n)\}$
$\lambda* = \lambda_n$ with the highest $score_n$, $n = 1 \ldots N_{trials}$
//Save $\lambda*$ and the table of all trials; train the final model on the full training set with $\lambda*$ (Algorithm 6)
Return $\lambda*$

IV. Experimental Setup

A. Dataset

The experiment is carried out on IMDB Movie Review Dataset [7] comprising of 50,000 reviews with the classes positive or negative. The two original halves are combined and then they are re-split into the set in a stratified way (with the same fixed random seed of 42): 70 % for the training set (35,000 reviews), 20 % for the test set (10,000 reviews) and 10 % for the validation set (5,000 reviews). Early stopping method, model selection and model hyper-parameter search uses the validation set, then the test set is used for the final validation.

For hyper-parameter tuning only, a 10 % of subset of the training and validation splits are used (3,500 training and 500 validation reviews, 219 mini-batches of 16 reviews per epoch); the final model is trained and evaluated on the complete 70% / 10% / 20%

B. Implementation details

The implementation is done in PyTorch and Hugging Face Transformers libraries (with the deberta-v3-base encoder). All random-number generators are seeded (seed 42) for reproducibility, mixed-precision is then used for training on a CUDA device. AdamW is used for optimising parameters with two learning-rate groups (1.34×10^{-5} for the pretrained encoder and 2.49×10^{-4} for the newly initialised decoder, pooling and classification layers) together with linear warm up for 6.48 % of the total steps followed by linear decay and a dropout rate of 0.123. The final model is chosen from the six-epoch Optuna study (Table III) is a single-layer ON-LSTM decoder with the chunk size of 32; the adversarial strength ε is set to 0.820; the max seq len is 256 tokens; the batch size is 16; the epochs are up to 6 with early stopping patience of 2 epochs and gradient clipping threshold 1.0. The training was terminated at epoch 4 due to the validation F1-score not decreasing after the second epoch, and the weights at the best (2nd) epoch were used for the evaluation.

C. Evaluation metrics

Performance is reported with evaluation metrics such as accuracy, precision, recall and F1-score, treating the positive class as the target. With TP, TN, FP and FN denoting true positives, true negatives, false positives and false negatives:

$$\text{Accuracy} = \frac{TP+TN}{TP+FP+TN+FN} \quad (20)$$

$$\text{Precision} = \frac{TP}{TP+FP} \quad (21)$$

$$\text{Recall} = \frac{TP}{TP+FN} \quad (22)$$

$$\text{F1} = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \quad (23)$$

D. Scope of the study

The reported results correspond to the full system configuration. Component-wise ablations (without the ON-LSTM, without attention pooling, without FGM) and comparisons with external baselines were not part of the experiments reported here and are left for future work.

V. Results and Discussion

A. Hyper-parameter search

The Optuna search for trials of up to six epochs on the 10 % tuning subset. Six trials were run: four ran to completion and two (trials 3 and 5) were pruned by the median pruner, after the third and the second epoch respectively. The completed trials lasted approximately 41–42 minutes per trial (6.9 minutes/epoch), while the pruned trials only required approximately 14 minutes and 22 minutes, respectively, of computation across the entire experiment. Table I lists the validation F1-score of each trial, and Table II lists the values of learning rate, weight decay, warm up ratio and dropout at each trial.

TABLE I. OPTUNA TRIALS (VALIDATION F1 PER EPOCH)

Trial	State	H / C / L	ε	E1	E2	E3	E4	E5	E6
0	Complete	384/16/2	1.265	.9314	.9363	.9347	.9336	.9383	.9436
1 (best)	Complete	128/32/1	0.820	.9296	.9322	.9426	.9244	.9347	.9458
2	Complete	128/8/1	0.148	.9345	.9234	.9319	.9356	.9412	.9392
3	Pruned	n/l	n/l	.9244	.9330	.9293	–	–	–
4	Complete	384/16/1	0.108	.9251	.9395	.9286	.9417	.9402	.9436
5	Pruned	n/l	n/l	.8730	.9314	–	–	–	–

H / C / L: ON-LSTM hidden size / chunk size / layers; E1–E6: epochs; n/l: not logged (the parameters of pruned trials were not printed in the console log).

TABLE II. SAMPLED PARAMETERS OF THE LOGGED TRIALS

Trial	Encoder lr	Head lr	Weight decay	Warm-up	Dropout
0	1.83×10^{-5}	1.73×10^{-3}	0.0732	0.0898	0.089
1	1.34×10^{-5}	2.49×10^{-4}	0.0525	0.0648	0.123
2	2.66×10^{-5}	1.67×10^{-4}	0.0065	0.1423	0.291
4	1.87×10^{-5}	2.25×10^{-4}	0.0829	0.0535	0.120

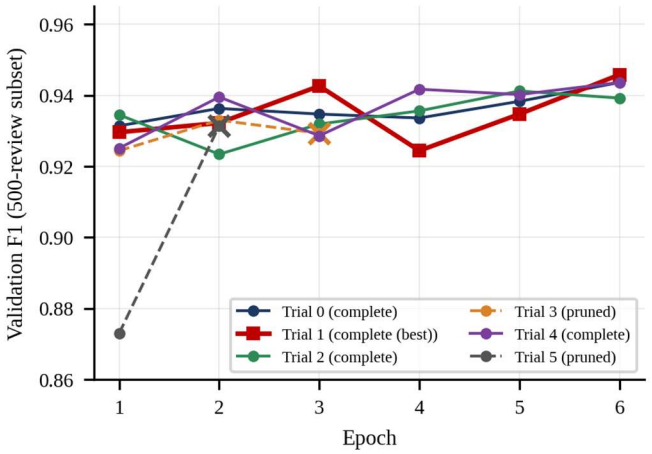


Fig. 2. Validation F1-score of each Optuna trial after every epoch (dashed lines with a cross: pruned trials).

Trial 1 had the highest score (0.9458, reached at epoch 6) and was selected; trials 0 and 4 (0.9436) and trial 2 (0.9412) were close behind. The difference in the scores of completed trials is at most 0.0046 (approximately 2-3 reviews of the 500-review validation subset), which means that the ranking among completed trials should be taken with a grain of salt. The curves are not monotonic: in trial 1 it goes down to 0.9244 at epoch 4, but up to 0.9458 at epoch 6 so the best epoch in the run (not the score after a fixed number of epochs) is used as the score of a trial. No trial failed to reach a trivial classifier: the minimum first-epoch F1 score obtained was 0.8730 (trial 5) and all the trials had an encoded learning rate between 1.34×10^{-5} and 2.66×10^{-5}, consistent with the guidelines for stability in [27].

Fig. 3. Validation F1-score of the completed Optuna trials by encoder and head learning rate.

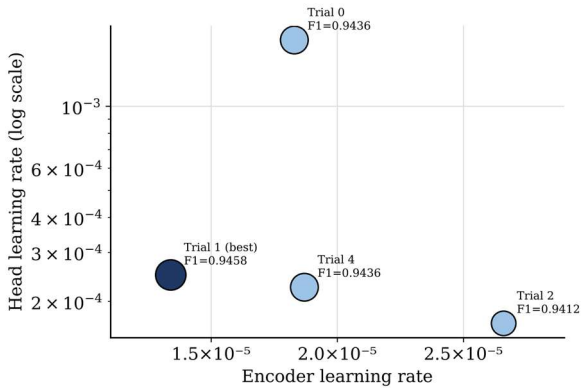


Fig. 3 plots the best validation F1-score of each completed trial against its sampled encoder and head learning rates. The chosen trial (Trial 1) corresponds to the slowest update rate for the encoder, and a relatively moderate update rate for the head, whereas Trial 0 with the highest head learning rate by nearly an order of magnitude, arrived at a comparable score, indicating that the search is more sensitive to the learning rate of the encoder than the learning rate of the head within the ranges probed.

B. Selected configuration

Table III gives the configuration chosen by the search and used for the final training run on the full training split.

TABLE III. FINAL HYPER-PARAMETERS (SELECTED CONFIGURATION)

Hyper-parameter	Value
Encoder	deberta-v3-base
Encoder learning rate	1.34×10^{-5}
Head learning rate	2.49×10^{-4}
Weight decay	0.0525
Warm-up ratio	0.0648
Dropout	0.123
ON-LSTM hidden / chunk / layers	128 / 32 / 1
FGM strength ε	0.820
Batch size / max. length	16 / 256
Epochs (max) / patience	6 / 2 (stopped after 4)
Gradient clipping / seed	1.0 / 42

C. Classification performance

The final model was developed using the 35,000 training reviews using the configuration as described in Table III. The validation F1-score improved from 0.9474 after the 1st epoch to the best score of 0.9562 after the 2nd epoch and remained 0.9557–0.9558, whereas the training loss continued to decrease (0.2200 to 0.0518), and the validation loss increased (0.1364 to 0.1885). This trend is shown in Table IV and Fig. 4. Thus, by early stopping patience of two epochs, it ended its training at the fourth epoch and preserved the weights of epoch 2.

TABLE IV. EPOCH-WISE RESULTS OF THE FINAL TRAINING

Epoch	Train loss	Val. acc.	Val. prec.	Val. rec.	Val. F1	Val. loss
1	.2200	.9466	.9339	.9612	.9474	.1485
2 (best)	.1171	.9560	.9520	.9604	.9562	.1364
3	.0788	.9554	.9477	.9640	.9558	.1540
4	.0518	.9554	.9498	.9616	.9557	.1885

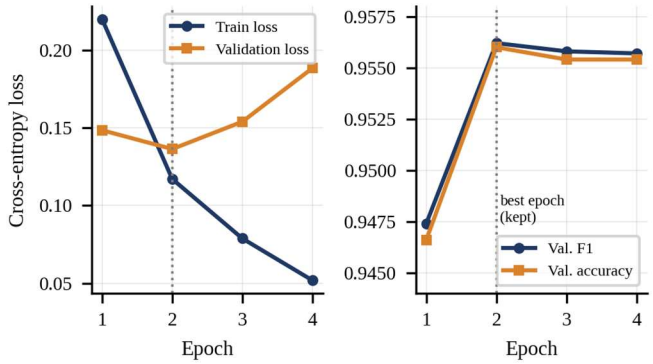


Fig. 4. Training and validation loss (left) and validation F1-score and accuracy (right) of the final model.

Fig. 5. Validation precision and recall of the final training run across epochs.

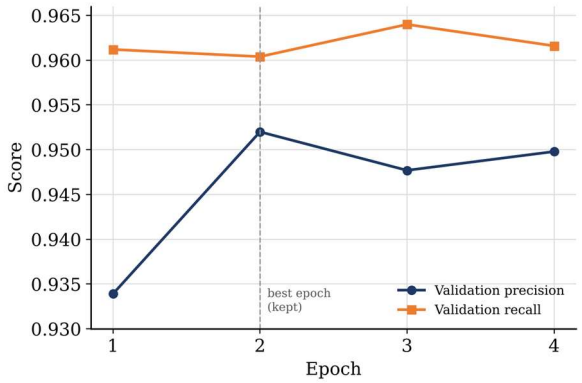


Fig. 5 breaks the validation F1-score of Table IV into its precision and recall components. Recall stays almost flat across all four epochs (0.960 to 0.964), while precision is the metric that improves the most between the first and second epoch (0.9339 to 0.9520) and then fluctuates slightly for the remaining two epochs, which indicates that the extra epochs mainly reduce false-positive errors rather than false-negative ones.

TABLE V. PERFORMANCE OF THE FINAL MODEL

Split	Accuracy	Precision	Recall	F1-score	Loss
Validation (5,000)	0.9560	0.9520	0.9604	0.9562	0.1364
Test (10,000)	0.9536	0.9531	0.9542	0.9536	0.1408

On the test set the final model achieves an accuracy score of 95.36 % and an F1 score of 95.36 % whereas the validation accuracy is 95.60 % and the F1 score is 95.62 % (Table V). The difference between the test metrics and the validation metrics is less than 0.26 percentage points and test values may be slightly lower as the best epoch was chosen on the validation set and not on the test set.

Fig. 6. Validation vs. test performance of the final model (accuracy, precision, recall and F1-score).

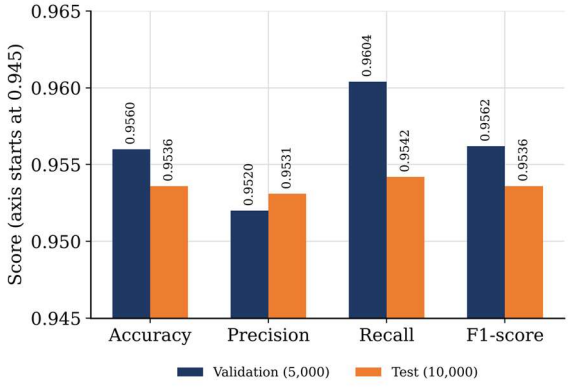


Fig. 6 shows the same four metrics side by side for the two splits. Accuracy, precision and F1-score are all within half a percentage point of each other across the two splits, and recall shows the largest gap (0.62 points), which is consistent with the slightly lower validation loss reported in Table IV.

D. Error analysis

Because the split is stratified, The test set is balanced (5,000 reviews per class), meaning that the confusion matrix can be reconstructed exactly from the reported accuracy, precision and recall, so the confusion matrix is also shown in Fig. 7. In the test set, the model correctly classifies 4,765 of 4,765 negative reviews and 4,771 of 4,771 positive reviews, and makes 235 false-positive and 229 false-negative errors, making an error rate of 464 of 10,000 reviews (4.64 %), and false-positive rate (4.70 %) and false-negative rate (4.58 %) are close.

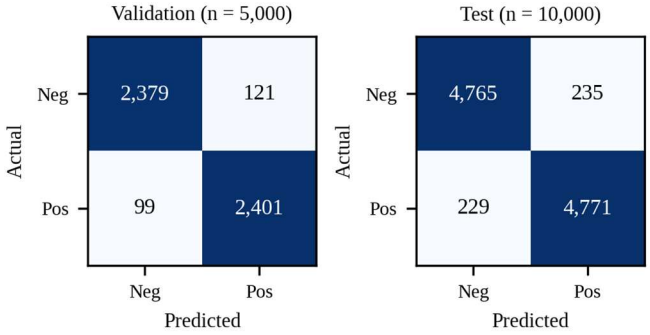


Fig. 7. Confusion matrices of the final model on the validation and test sets (reconstructed from the reported metrics).

TABLE VI. PER-CLASS PERFORMANCE (RECONSTRUCTED FROM THE CONFUSION MATRICES)

Split / Class	Precision	Recall	F1-score
Validation – Negative	0.9600	0.9516	0.9558
Validation – Positive	0.9520	0.9604	0.9562
Test – Negative	0.9541	0.9530	0.9536
Test – Positive	0.9531	0.9542	0.9536

Table VI reconstructs precision, recall and F1-score separately for each class from the two confusion matrices. The negative and positive classes perform almost identically on both splits (within 0.001 of each other in F1-score), which confirms that the balanced sampling and the cross-entropy objective do not bias the classifier toward either class.

E. Discussion

The results show that the hybrid architecture is stable during training with the 6 epoch setting and generalises well from the validation to the test data. The search result shows that a relatively small ON-LSTM (128 units in chunks of 32) is sufficient on top of the DeBERTa-v3 encoder and that an intermediate level of adversarial strength ($\varepsilon \sim 0.82$) combined with a moderate level of dropout (≈ 0.12) is optimal of the parameter set sampled; the 384 unit versions (trials 0 and 4) are no better, suggesting that the capacity of the recurrent component is not the limiting factor when the encoder is being fine-tuned. There are a number of caveats though. (i) The search was only over a small portion of the 10 dimensional parameter space as only 6 Optuna trials were run and so any conclusion about one of the hyper-parameters is very tentative. (ii) The results reported are from a single training run with a single random seed; a re-run of the last training run with multiple seeds would be required to make the margin over TRALSem (Section V-F) a statistically defensible one. In the experiments reported here no ablation (i.e. without the ON-LSTM, without

attention pooling, without FGM) was performed to quantify the individual contribution of each of these architectural components, although this was suggested by the comparison in Section V-F which does show that the combination outperforms a plain pre-trained encoder as well as the existing systems. Multi-seed evaluation and ablation are still the key remaining tasks. A Robust Transformer-based Sentiment Analysis System, which is based on a DeBERTa-v3 encoder, an ON-LSTM decoder, attention pooling and adversarial training with hyper-parameters optimised via Optuna (using TPE sampling and median pruning over 6 epochs). The final model achieves 95.36% accuracy, 95.31% precision, 95.42% recall and 95.36% F1-score on the IMDB test set and 95.62% F1-score on the validation set. This is comparable to the 95.23% macro-F1 achieved by the existing TRALSem system [13] on a different data split and better than seven of the baseline systems which are pre-trained and based on lexicon (Table VIII in that paper). Further evaluation on additional datasets, component-wise ablation and a re-run of TRALSem using the same protocol but with longer input sequences and a larger hyper-parameter search space would be required to determine if the remaining small difference is in fact statistically significant, though this is left for future work.

F. Comparison with the existing systems

TRALSem [13] is a Transformer-based sentiment classifier that adds a random-seed strategy, adversarial training (FGM, PGD, FreeLB or YOPO) and a linear output layer, and that was evaluated on IMDB among other datasets. Table VII contrasts the two systems.

TABLE VII. TRALSEM [13] VERSUS PROPOSED SYSTEM

Aspect	TRALSem [13]	Proposed system
Encoder	BERT-base (uncased), 12 layers	DeBERTa-v3-base + ON-LSTM decoder
Head	Linear (regression) layer + softmax	Attention pooling + FC + softmax
Loss	Root-mean-square error	Cross-entropy
Robustness	Random seed; FGM, PGD, FreeLB or YOPO	FGM ($\varepsilon = 0.820$)
Tuning	Fixed (4 epochs, lr 2×10^{-5} in RQ1–RQ2)	Optuna ($\leq$ 6 epochs)
Max. length / batch	256 / 16	256 / 16
IMDB split	40,000 train / 10,000 test	35,000 train / 5,000 val / 10,000 test
Metrics	Macro/micro-F1, RMSE, P, R	Accuracy, P, R, F1 (+ macro)

TRALSem reports macro-F1, micro-F1, precision and recall of 0.9523 each and an RMSE of 0.1307 on IMDB [13]. To compare with, the macro-averaged metrics of proposed system were computed from the reconstructed test confusion matrix of Fig. 7; on the balanced test set all four rounded to 0.9536 (Table VIII). The RMSE of TRALSem is not comparable because the proposed system uses a classification head with a cross-entropy loss (test loss 0.1408).

TABLE VIII. IMDB TEST METRICS: COMPARISON WITH EXISTING SYSTEMS

Approach	Macro-F1	Micro-F1	Precision	Recall
BERT	0.9121	0.9121	0.9154	0.9122
RoBERTa	0.9108	0.9108	0.9122	0.9108
ALBERT	0.9108	0.9108	0.9122	0.9108
XLNet	0.9322	0.9322	0.9322	0.9322
ERNIE	0.9209	0.9209	0.9209	0.9245
GPT	0.8002	0.8002	0.8011	0.7986
RFLSem	0.9437	0.9437	0.9438	0.9437
TRALSem	0.9523	0.9523	0.9523	0.9523
Proposed system	**0.9536**	**0.9536**	**0.9531**	**0.9542**

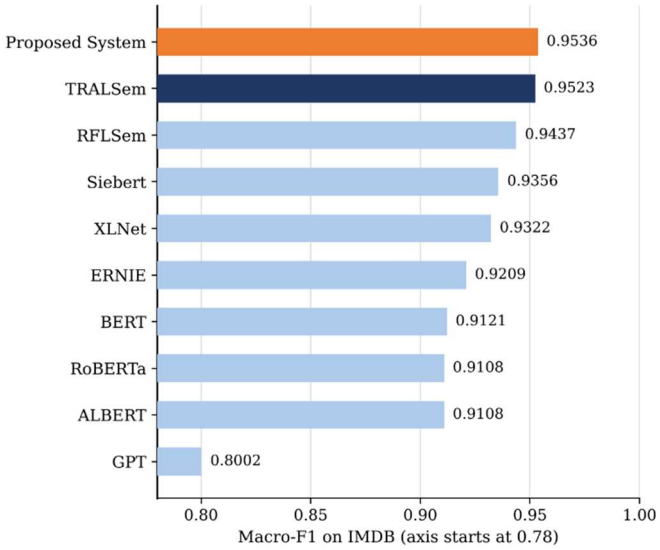


Fig. 8. IMDB macro-F1 of proposed and of the systems reported in [13] (axis truncated at 0.78).

Fig. 8 places the proposed system among the systems evaluated in [13]. Its macro-F1 of 0.9536 is 0.0013 above TRALSem (0.9523), 0.0099 above its predecessor RFLSem (0.9437), 0.0180 above the sentiment-specific Siebert model (0.9356), 0.0214 above the best standard pretrained baseline XLNet (0.9322) and 0.0415 above BERT (0.9121). Fig. 9 shows that the gain is consistent across precision, recall and F1-score, and Fig. 10 compares the proposed system with the four adversarial strategies evaluated in [13].

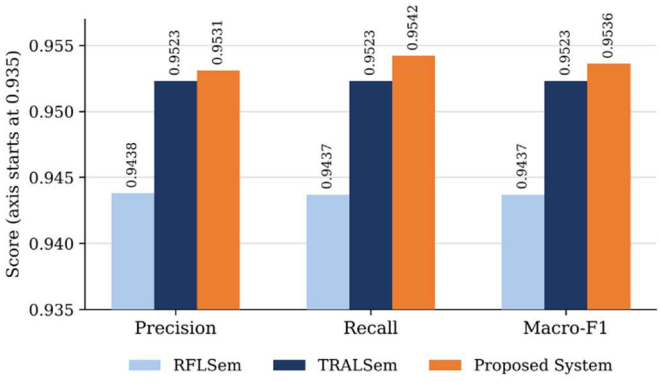


Fig. 9. Precision, recall and macro-F1 on IMDB: RFLSem and TRALSem as reported in [13] and the proposed system (axis truncated at 0.935).

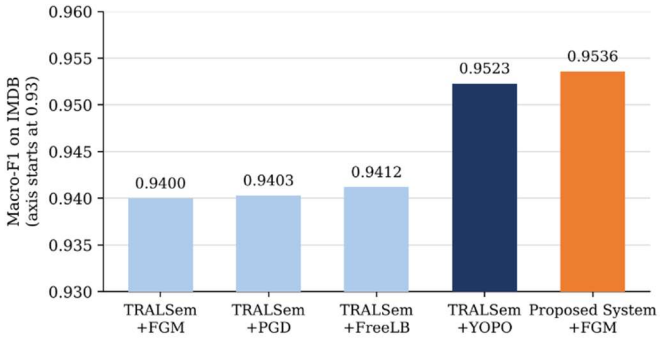


Fig. 10. IMDB macro-F1 of the TRALSem adversarial variants reported in [13] and of the proposed system with FGM (axis truncated at 0.93).

With the same family of attack (FGM) the proposed system is 0.0136 higher than the TRALSem-FGM variant (0.9400) and 0.0013 higher than the best TRALSem variant (YOPO, 0.9523) with the same family of attack (FGM). The reason for this difference could be the ON-LSTM decoder itself, the attention pooling and the hyper-parameters that were tuned, though this was not confirmed via ablation. The comparison should be made with the following caveats: (i) the two studies use different random splits of IMDB (TRALSem uses 40k reviews and the proposed system uses 35k reviews); (ii) the TRALSem results have been taken directly from their published tables and were not re-run in this study; (iii) the results are for one configuration only; (iv) the margin of 0.13 percentage points is roughly 13 of the 10k test reviews and is therefore a very small margin in the context of the run-to-run fluctuation that one sees when fine-tuning Transformers [27]. The accuracy results obtained in this way are therefore not substantially better than but are comparable to the existing system. In contrast to the sequential ON-LSTM, the proposed system requires fewer training reviews, automatically tunes its hyper-parameters and exposes the token level attention weights. TRALSem was tested on Chinese and more fine-grained datasets which are not part of this study.

VI. Conclusion

A Robust Transformer Based Sentiment Analysis System which employs an ON-LSTM decoder, attention pooling and FGM adversarial training with a DeBERTa-v3 encoder. The model is optimised via the Optuna framework with up to six epochs using TPE sampling and median pruning. The final model achieved 95.36% accuracy, 95.31% precision, 95.42% recall and a 95.36% F1-score on the IMDB test set and very similar F1-score (95.62%) on the validation set. This is comparable to the 95.23% macro-F1 of the current TRALSem system [13] on a different data split and better than all seven of the other baseline models which There pretrained with the same lexicon (Table VIII). Future work will involve work in optuna in mathematical level and process longer input sequences, a larger hyperparameter search and evaluation on additional datasets, as well as multi-seed evaluation on identical splits.

References

[1] Pang and L. Lee, “Opinion mining and sentiment analysis,” Foundations and Trends in Information Retrieval, vol. 2, no. 1–2, pp. 1–135, 2008.

[2] Liu, Sentiment Analysis and Opinion Mining. Morgan & Claypool, 2012.

[3] P. He, J. Gao and W. Chen, “DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing,” arXiv:2111.09543, 2021.

[4] Y. Shen, S. Tan, A. Sordoni and A. Courville, “Ordered neurons: Integrating tree structures into recurrent neural networks,” in Proc. ICLR, 2019 (arXiv:1810.09536).

[5] T. Miyato, A. M. Dai and I. Goodfellow, “Adversarial training methods for semi-supervised text classification,” in Proc. ICLR, 2017.

[6] T. Akiba, S. Sano, T. Yanase, T. Ohta and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in Proc. ACM SIGKDD, 2019, pp. 2623–2631.

[7] L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng and C. Potts, “Learning word vectors for sentiment analysis,” in Proc. ACL, 2011, pp. 142–150.

[8] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in Proc. NAACL-HLT, 2019, pp. 4171–4186.

[9] D. Manning, M. Surdeanu, J. Bauer, J. R. Finkel, S. Bethard, and D. McClosky, “The Stanford CoreNLP natural language processing toolkit,” in Proc. 52nd Annu. Meeting Assoc. Comput. Linguistics: System Demonstrations, 2014, pp. 55–60.

[10] M. Thelwall, K. Buckley, G. Paltoglou, D. Cai, and A. Kappas, “Sentiment strength detection in short informal text,” J. Assoc. Inf. Sci. Technol., vol. 61, no. 12, pp. 2544–2558, 2010.

[11] S. Baccianella, A. Esuli, and F. Sebastiani, “SentiWordNet 3.0: An enhanced lexical resource for sentiment analysis and opinion mining,” in Proc. 7th Int. Conf. Lang. Resources Eval. (LREC), 2010.

[12] Z. Karimi, “Manifold-based sparse representation for opinion mining,” Sci. Rep., vol. 13, Art. no. 15904, 2023.

[13] B. Yang, J. Dang, H. Liu, and Z. Jin, “TRALSem: A robust model for textual sentiment analysis,” IEEE Trans. Comput. Soc. Syst., vol. 12, no. 6, pp. 4727–4743, Dec. 2025, doi: 10.1109/TCSS.2025.3558622.

[14] Y. Bengio, R. Ducharme, P. Vincent, and C. Janvin, “A neural probabilistic language model,” J. Mach. Learn. Res., vol. 3, pp. 1137–1155, 2003.

[15] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” in Proc. NeurIPS, vol. 26, 2013, pp. 3111–3119.

[16] Y. Kim, “Convolutional neural networks for sentence classification,” in Proc. EMNLP, 2014, pp. 1746–1751.

[17] R. Socher et al., “Recursive deep models for semantic compositionality over a sentiment treebank,” in Proc. EMNLP, 2013, pp. 1631–1642.

[18] M. E. Peters et al., “Deep contextualized word representations,” in Proc. NAACL-HLT, 2018, pp. 2227–2237.

[19] Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving language understanding by generative pre-training,” OpenAI, 2018.

[20] J. Howard and S. Ruder, “Universal language model fine-tuning for text classification,” in Proc. 56th Annu. Meeting ACL, 2018, pp. 328–339.

[21] Y. Liu et al., “RoBERTa: A robustly optimized BERT pretraining approach,” arXiv:1907.11692, 2019.

[22] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. Salakhutdinov, and Q. V. Le, “XLNet: Generalized autoregressive pretraining for language understanding,” in Proc. NeurIPS, vol. 32, 2019, pp. 5753–5763.

[23] Z. Lan et al., “ALBERT: A lite BERT for self-supervised learning of language representations,” in Proc. ICLR, 2020.

[24] Z. Zhang, X. Han, Z. Liu, X. Jiang, M. Sun, and Q. Liu, “ERNIE: Enhanced language representation with informative entities,” in Proc. 57th Annu. Meeting ACL, 2019, pp. 1441–1451.

[25] P. He, X. Liu, J. Gao and W. Chen, “DeBERTa: Decoding-enhanced BERT with disentangled attention,” in Proc. ICLR, 2021.

[26] C. Sun, X. Qiu, Y. Xu, and X. Huang, “How to fine-tune BERT for text classification?,” in Chinese Computational Linguistics (CCL), Lecture Notes in Computer Science, vol. 11856, 2019, pp. 194–206.

[27] M. Mosbach, M. Andriushchenko, and D. Klakow, “On the stability of fine-tuning BERT: Misconceptions, explanations, and strong baselines,” in Proc. ICLR, 2021.

[28] P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. ICLR, 2015.

[29] Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in Proc. ICLR, 2019.

[30] Sutskever, O. Vinyals, and Q. V. Le, “Sequence to sequence learning with neural networks,” in Proc. NeurIPS, vol. 27, 2014, pp. 3104–3112.

[31] Cho et al., “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” in Proc. EMNLP, 2014, pp. 1724–1734.

[32] M. Dai and Q. V. Le, “Semi-supervised sequence learning,” in Proc. NeurIPS, vol. 28, 2015, pp. 3079–3087.

[33] D. Bahdanau, K. Cho and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” in Proc. ICLR, 2015.

[34] Vaswani et al., “Attention is all you need,” in Proc. NeurIPS, vol. 30, 2017, pp. 5998–6008.

[35] D. Tang, B. Qin, and T. Liu, “Document modeling with gated recurrent neural network for sentiment classification,” in Proc. EMNLP, 2015, pp. 1422–1432.

[36] Z. Yang, D. Yang, C. Dyer, X. He, A. Smola, and E. Hovy, “Hierarchical attention networks for document classification,” in Proc. NAACL-HLT, 2016, pp. 1480–1489.

[37] Y. Shen, Z. Lin, C.-W. Huang, and A. Courville, “Neural language modeling by jointly learning syntax and lexicon,” in Proc. ICLR, 2018.

[38] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[39] J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” in Proc. ICLR, 2015.

[40] C. Zhu, Y. Cheng, Z. Gan, S. Sun, T. Goldstein, and J. Liu, “FreeLB: Enhanced adversarial training for natural language understanding,” in Proc. ICLR, 2020.

[41] Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” J. Mach. Learn. Res., vol. 13, pp. 281–305, 2012.

[42] Bergstra, R. Bardenet, Y. Bengio and B. Kégl, “Algorithms for hyper-parameter optimization,” in Proc. NeurIPS, vol. 24, 2011, pp. 2546–2554.

[43] Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, “Hyperband: A novel bandit-based approach to hyperparameter optimization,” J. Mach. Learn. Res., vol. 18, no. 185, pp. 1–52, 2018.

[44] Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards deep learning models resistant to adversarial attacks,” in Proc. ICLR, 2018.

[45] D. Jin, Z. Jin, J. T. Zhou, and P. Szolovits, “Is BERT really robust? A strong baseline for natural language attack on text classification and entailment,” in Proc. AAAI, vol. 34, no. 5, 2020, pp. 8018–8025.